The tools and techniques of R programming
Lesson-6
User-defined functions in R

By
Prof. Dr. A.B.Chowdhury,HOD,CA

Techno India University, West Bengal,India
For any query,Email-Id:hod.ca@technoindiaeducation.com

August 28, 2023

Objective of this Lesson

This lesson aims **to provide with the idea of modular programming in R**. As structured and modular programming forms the foundation of a good software system, we should learn to write such programs. In R, a module can be implemented as a function. Functions defined by users are reusable in different programs thus reducing the time and labour of software development. Hence, **the importance of learning to develop user-defined functions is multifarious**. **Prerequisite:** Knowledge of a programming language

The Modular Programming through functions in R

R allows users to define functions of their choice. The user-defined functions enable users to develop modular programs. This is because, a function, is a self-contained unit of code for serving a specific purpose and such a set of codes is known as a module in System Design. The codes in a function should be such that it solves a particular part of a program in a generalized form in order to become re-usable in identical situations and it can return some value, if required, to the calling point after the execution. In R, a user-defined function may also replace the use of existing available functions in the library of the system. This may give a better satisfaction to the programmer. It will also prove a deep understandability of the coder regarding the program logic. Above all, functions enable us to adopt the top-down design approach by breaking a large complex program into small manageable modules that can be handled by different people separately and then the modules can be integrated to execute together. It also makes the testing of the codes an easier task leading to the development of easily understandable, better readable and easily debuggable programs.

The syntax of a user-defined function in R

The values that we pass to a function for processing are called arguments to the function. A function may or may not need arguments. In R, a user-defined function is coded according to the following syntax:

User-defined–function-name Assignment-operator function([list of arguments]) { **body of the codes in the function**}

Here, **function is a R key-word**. **The list of arguments is the values being passed to the function separated by comma**. **By showing the list within square brackets, we imply that their use is optional. During the use of arguments, the square brackets must not be used.** After defining a function, **it is called by using the function name and the arguments within brackets**. The **'return'** statement in a function returns the the desired value from a function.Let us have some illustrative examples of user-defined functions.

Illustrative Example 6.1 on User-defined functions in R

Example-6.1: Let us develop a function to return the absolute value of any number.

Task Analysis: The absolute value of a number is its magnitude without any regard to its sign. Hence, the absolute value of a positive number will be the number itself and that of a negative number will be its positive counterpart.

The following R-script shows the required code:

```
# Function for obtaining the absolute value
absolute=function(x){
if(x>0){
x=-x
}
return(x)
}
```

Illustrative Example 6.2 on User-defined functions in R

Example-6.2. It is required to develop a function to obtain the factorial value of a number.

Task Analysis: Determination of factorial of any given number is a very simple and common task. We know that the factorial of any whole number ,including zero, is only defined and it is the product of all natural numbers from 1 to that number. Factorial zero is defined to be 1.

The following R-code of shows the function:

```
#function to obtain the factorial of a number
fact=function(n){
if(n==0)
return(1)
p=1
while(n>0){
p=p*n
n=n-1
}
```

Illustrative Example 6.3 on User-defined functions in R

Example-6.3. A function is required to be developed for evaluating an exponential expression of the form a^b, where 'a' is the base and 'b' is any power of a.

Task analysis. We know that a raised to the power b can be obtained by multiplying a to itself b times. So, the task is very easy as demonstrated in the R-code below:

```
power=function(a,b){
 p=1
 while(b>0){p=p*a
 b=b-1}
 return(p)
}
```

Alternatively, we can make use the exponential operator ** and develop the function in the following way:

```
 pow=function(a,b){
 r=a**b
return(r)
 }
```

Illustrative Example 6.4 on User-defined functions in R

Example-6.4.A function is required to be developed for returning the nth **Fibonacci number**.

Task Analysis: We know that the Fibonacci numbers form a series such that any term of the series can be obtained by taking the sum of the preceding two terms where the first two terms are 0 and 1. Keeping this concept in mind, we develop the following function:

```
fibo=function(n){
 t1=0
 t2=1
 m=1
while(m<n){
 t=t1+t2
 t1=t2
 t2=t
 m=m+1
 }
return(t1)
 }
```

Illustrative Example 6.5 on R User-defined functions

Example-6.5. It is required to develop a function in order to check whether a given string is a **palindrome** or not.

Task Analysis: A given string is said to be a **palindrome** if the reversed form of the string equals the given string. The reversed form can be obtained by using the **substr() function** from the last to the first character as shown in the Code below:

```
# Testing for a Palindrome
str=readline(prompt="Enter any string for testing: ")
l=nchar(str)
rstr=""
for(i in seq(l,1,-1)){
s=substr(str,i,l);l=l-1
rstr=paste(rstr,s,sep="",collapse=NULL)}
if(str==rstr){
print("It is a palindrome")
} else {
print("It is not a palindrome")}
```

Illustrative Example 6.6 on R User-defined functions

Example-6.6. It is required to develop a function for obtaining the square root of a real number.

Task analysis: To obtain the square root of a real number, we may make use of the **Newton Raphson method**. Here, we assume that the square root of any number is 1 which is nothing but our initial guess. We then determine the absolute value of the difference between the square of the guess and the number; if the guessed value happens to be the square root value, then the difference will become zero obviously, as for 1. It happens particularly for numbers which are perfect squares; otherwise the difference will either be a large value or a value which is very small that can be neglected for numbers which are not perfect squares. The magnitude of the neglected value may be less than that of a very small predefined value, however small it is so that the current value of guess can be accepted as the square root of the number under consideration; otherwise, we update the value of guess by using the formula: guess$=\frac{(guess+\frac{x}{guess})}{2}$ and then repeat the process. The algorithm is given below for any real number x.

Illustrative Example 6.6 on R User-defined functions–Contd.

1. **Set guess to 1**
2. **Set epsilon to 0.0001, the small predefined value.**
3. **If** $|guess^2 - x|$ **<epsilon then go to step 6.**
4. **Guess=**$\frac{(guess+\frac{x}{guess})}{2}$
5. **Go to step 3**
6. **Guess is the square root of x**
7. **End**

The following R script serves the purpose:

```
 sqrtt=function(x){
guess=1.0
 epsilon=.00001
while(abs(guess*guess-x)>epsilon){
 guess=(guess+x/guess)/2
}
return(guess)
 }
```

Illustrative Example 6.7 on R User-defined functions

Example-6.7. It is required to develop a function in R to obtain the square root of a complex number without making use of any library function. **Task Analysis:** we know that the formula for finding square root of a complex number is: $\sqrt{a+bi} = \pm(\sqrt{\frac{|z|+a}{2}} + i\frac{b}{|b|}\sqrt{\frac{|z|-a}{2}})$

Where, z=modulus of the complex number,a=real part of the complex number and b is the imaginary part of the complex number. In R , we utilize **Mod()** for the modulus, **Re()** for the real part and **Im()** for the imaginary part of a complex number. Here, the functions for determining the square root and absolute value of a real number will also be required. The developed functions are as shown below:

```
sqrtt=function(x){
 guess=1.0
 epsilon=.00001
while(absolute(guess*guess-x)>epsilon){
guess=(guess+x/guess)/2
 }
 return(guess)
 }
```

Illustrative Example 6.7 on R User-defined functions–Contd.

```
 absolute=function(x){
 if(x<0)
 x=-x
 return(x)
}


complex_root=function(c){
 z=Mod(c)
 r=Re(c)
 i=Im(c)
if(i==0){
srt=complex(real=0,imaginary=1)
 } else {
mod_b=absolute(i)
 p=i/mod_b
p1=(z+r)/2;p2=p*(z-r)/2
 x=sqrtt(p1);y=sqrtt(absolute(p2))
 if(p<0)
 y=-y
 srt=complex(real=x,imaginary=y)
 }
 return(srt)
 }
```

Lazy Evaluation

The term **lazy evaluation** refers to a characteristic of R functions in which the arguments passed to an R function are evaluated only when they are needed within the function. So, even if some passed argument except the first one is not used within a function, R will execute the function without giving any error. Let us consider the following function and some call to it.>**f=function(a,b)print(a)** >**f(3)**
[1] 3
So, we observe that the above function does not generate any error as no reference to the variable b is done within the function. The value passed to the function positionally matches with the first argument a.Now, let us study another function version of the function and its execution as follows:
>**f=function(a,b){print(b)}** >**f(2)**
Error in print(b) : argument "b" is missing, with no default
Let us study another version of the function:
>**f=function(a,b){print(a);print(b)}**
>**f(9)**
[1] 9
Error in print(b) : argument "b" is missing, with no default
It may be observed that the error in the execution of the above function is thrown after printing the value of a; because, it was not referenced before that.

Recursive functions in R

R supports **recursion in functions which is calling the function itself until a given base condition is satisfied**. As recursion creates a stack to store the intermediate values that can be processed speedily through LIFO method; it saves both time and space. Hence, recursion is always preferable to iteration.

Features of Recursion

- The use of recursion, often, makes code shorter and looks cleaner.
- It is a Simple solution for a few cases.
- It expresses in a self-calling function .

Applications of Recursion

Recursions are used extensively in the following programming issues:

A. Dynamic Programming

It is the process of avoiding recomputation. It is an essential tool for statistical programming. There are two types of dynamic programming:

1. **Bottom-up dynamic programming**
   - In this, we check function starting with the smallest possible argument value.
   - All computed values will be stored in an array.

Applications of Recursion–Contd. and Example-6.8

2. **Top-down dynamic programming**
   - Save each computed value as the final act of a recursive function.
   - Check if pre-computed values exist as the first action.

B. Divide-And-Conquer Algorithms

- It is a Common class of recursive function.
- Common features of this algorithm are process inputs, divide the input into Smaller portions, Recursive call(s) process at least one part.
- Recursion may sometimes occur before the input has been processed.

We shall now illustrate some recursive functions.

Example-6.8. It is required to develop a recursive function evaluation of factorial of a number.

The recursive R-script is shown below:

```
# Factorial through recursion
fact=function(n){
if(n==1||n==0)
return(1)# Base condition required for stopping the recursive call
return(n*fact(n-1))}# Recursive call
print("End of program....Thank You")
```

Illustrative Examples-6.9 and 6.10 on R Recursive functions

Example-6.9. It is required to develop a recursive function for evaluating the GCD (Greatest Common Divisor or HCF) of any two given numbers.

Task Analysis: The logic behind was explained in the lesson on iteration. The recursive R function is as shown below:

```
# GCD through recursion
GCD=function(a,b){
if(a%%b==0)
return(b) # base condition
return(GCD(b,a%%b))}
```

Example-6.10. It is required to develop a recursive function to return the nth **Fibonacci number**.

Task Analysis: The logic behind was explained in the lesson on iteration. The required code is shown below:

```
# Recursion for the nth Fibonacci number
fibo=function(n){
if(n≤0)
stop(" Invalid Term",call.=TRUE) # This is to cancel execution of the function with ERROR message
if(n==1){
return(0)
}else if(n==2){
return(1)
}
return(fibo(n-1)+fibo(n-2))
}
```

Illustrative Examples-6.11 and 6.12 on R Recursive functions

Example-6.11. It is required to develop a recursive function to convert a decimal number into its equivalent binary one.

Task Analysis: The conversion is done mathematically by dividing the decimal number by 2, the base of the binary number system and then collecting and showing the remainders in the reverse sequence.The required function is shown below:

```
# Program to convert decimal number into binary number using recursive function
binary = function(n) {
if(n >1) {print(n%%2)
binary(as.integer(n/2))
}
cat(n %% 2)
}
```

Example-6.12. It is required to develop a recursive function to evaluate the sum of the following series:

2+4+6+8+.....+2n, where n is any whole number.

Task Analysis: Let S(n) denote the sum of n terms of the series. So, s(0)=0, s(1)=2, s(2)=2+4=6, so on. It is easily understood that the recursive function will be: s(n)=s(n-1) +2n, n≥1. The R script is now shown below:

Illustrative Examples-6.12 and 6.13 on R Recursive functions

```
# Function to determine sum ofa given series
sum_series=function(n){
if(n==0)
return(0)
return(sum_series(n-1)+2*n)
}
```

Example-6.13. A robot can take strides in two ways: either a stride of 1 foot or a stride of 2 feet. It is required to develop a recursive function to determine the number of ways in which the robot can cross n feet.

Task Analysis: It may be observed that for n=1, the number of ways=1; for n=2, the number of ways=2(1,1 and 2), for n=3, the number of ways=3(1,1,1; 2,1;1,2); for n=4, the number of ways=5, because, 5=(1,1,1,1,1 or 1,1,1,2 or 2,1,1,1 or 2,2,1 or 1,2,2). An interesting observation is:

Way(1)=1
Way(2)=2
Way(3)=way(1) + way(2)
Way(4)=way(2) +way(3) and so on.
Thus, way(n)=way(n-1) +way(n-2).

The following is the requisite R script for the determination of the desired number of ways:

Example 6.13 Contd.

```
# Finding out the number of ways a robot may cross a distance
robo_ways=function(n){
if(n≤0)
stop("Invalid Distance")
if(n==1||n==2)
return(n)
return(robo_ways(n-1)+robo_ways(n-2))
}
```

Example-6.14. One version of **Ackermann's function** is defined as follows:

$$Ack(M,N) = \begin{cases} \text{N+1,if M=0} \\ \text{ack(M-1,1),ifM0 and N=0} \\ \text{ack(M-1,ack(M,N-1)),if M}\neq 0 \text{ and N} \neq 0 \end{cases} \quad (1)$$

It is required to develop a recursive function that computes Ackermann's function.

Illustrative Example-6.14 on R Recursive functions–Contd.

Task Analysis: In computability theory, the Ackermann function, named after Wilhelm Ackermann, is one of the simplest and earliest-discovered examples of a total computable function that is not primitive- recursive. All primitive- recursive functions are total and computable, but the Ackermann function illustrates that not all total computable functions are primitive- recursive.
After Ackermann's publication of his function (which had three nonnegative integer arguments), many authors modified it to suit various purposes, so that today "the Ackermann function" may refer to any of numerous variants of the original function. Our version, the two-argument Ackermann function, is defined as stated in the problem.: The following R-script the function:

```
# Function for the evaluation of Ackermann's function.
ack=function(m,n){
if(m==0)
return(n+1)
if(n==0&&m!=0)
return(ack(m-1,1))
if(m!=0&&n!=0)
return(ack(m-1,ack(m,n-1)))}
```